

Informe de Pruebas

Proyecto Requintu — Backend y Frontend (E2E)

47 pruebas ejecutadas — 47 aprobadas (100%)

Backend: Node.js (node:test) — 34 pruebas

Frontend E2E: Playwright + Chromium — 13 pruebas

Fecha: 18 de septiembre de 2026

Versión 1.4

1. Resumen

Se ejecutaron las pruebas automatizadas del proyecto Requintu, tanto del backend (API REST) como del frontend (flujo completo del usuario en un navegador real). Todas las pruebas pasaron.

Cantidad	Tipo
34	Pruebas backend (API)
13	Pruebas e2e frontend
47	Total superadas

En versiones anteriores se detectó y corrigió el problema de la subida de imágenes que aceptaba archivos que no eran imágenes reales (ahora la API valida la estructura real con magic bytes y responde 400) y se añadieron la moderación NSFW y el sistema de denuncias. En la v1.3 se agregaron esas pruebas. En esta versión (v1.4) se añadieron: pruebas de los roles comerciante_premium (límite de 5 locales y galería de imágenes) y alcaldía (local destacado primero en la búsqueda), pruebas de seguridad derivadas de la auditoría — F01 (SSE autenticado con límites), F02 (cuota diaria de subidas) y F03 (allowlist de imagen_url) — y una prueba e2e de regresión de sesión (la sesión persiste al recargar con la cookie httpOnly y credentials: include).

2. Programas y herramientas utilizados

Herramienta	Versión	Uso en las pruebas
Node.js	v25.8.0	Ejecución del backend, del runner de pruebas y de los scripts.
npm	11.11.0	Instalación de dependencias y ejecución de scripts de prueba.
node:test (runner nativo de Node)	—	Framework de pruebas del backend: node --test.
Playwright (@playwright/test)	1.62.1	Framework de pruebas e2e del frontend en un navegador real.
Chromium (headless)	Chrome Headless Shell 151.0.7922.34	Navegador automatizado donde corren las pruebas e2e.
Vite / rolldown	8.x	Compilación de producción del frontend y servidor de preview para las e2e.
Express / mysql2	4.x / 3.x	Servidor de la API y acceso a la base de datos durante las pruebas.
MySQL (TiDB Cloud)	MySQL 8	Base de datos real sobre la que se hicieron las pruebas de integración.
Cloudinary	API v2	Servicio externo usado en la prueba de subida de imágenes.
nsfwjs + @tensorflow/tfjs (puro JS)	2.x / 4.x	Clasificador local (MobileNetV2) que modera imágenes dentro de la propia API.
Brevo (correo)	API v3	Flujo de recuperación de contraseña; el envío pasa por la cola de correos.

Herramienta	Versión	Uso en las pruebas
bcryptjs / jsonwebtoken	2.x / 9.x	Cifrado de contraseñas y generación de tokens verificados por las pruebas.

3. Cómo se ejecutan las pruebas

3.1 Backend (34 pruebas)

La suite levanta el servidor Express en memoria (puerto aleatorio), se conecta a la base de datos real (TiDB Cloud / local), crea usuarios y publicaciones de prueba y los elimina al terminar. Los correos del flujo de recuperación se encolan (no se espera su envío durante la prueba).

```
npm test # cd del directorio raíz del proyecto
# Resultado: 34 passed, 0 failed (~5-8 s)
```

3.2 Frontend e2e (13 pruebas)

Playwright compila el frontend (npm run build), arranca el servidor de preview (puerto 4173) y el backend local (puerto 3000), y abre un Chromium headless que recorre la interfaz como lo haría un usuario.

```
cd frontend
npx playwright install chromium # solo la primera vez
npm run test:e2e
# Resultado: 13 passed, 0 failed (~15-40 s)
```

4. Pruebas del backend por caso

#	Prueba	Qué verifica	Resultado
1	rechaza registro con email invalido	Validación de entrada (express-validator) devuelve 400.	Aprobada
2	rechaza registro con password corta	La contraseña debe tener mínimo 8 caracteres (400).	Aprobada
3	convierte intento de rol admin en turista	Nadie puede auto-registrarse como admin.	Aprobada
4	login responde identico con usuario inexistente y password erronea	No se filtra qué credencial falló (anti-fuerza bruta).	Aprobada
5	ruta protegida sin token devuelve 401	authMiddleware bloquea sin token.	Aprobada
6	token falsificado es rechazado con 403	JWT firmado con otro secreto no es válido.	Aprobada
7	perfil responde datos del usuario autenticado	GET /api/perfil devuelve email y rol correctos.	Aprobada
8	comerciante no puede crear publicaciones	Función de roles (403 para comerciante).	Aprobada
9	comerciante solo puede crear un local	Límite de locales por rol (1 para comerciante); el segundo intento responde 403.	Aprobada
10	rol comerciante_premium no puede elegirse en el registro	El registro solo acepta turista/comerciante; el intento devuelve 400.	Aprobada
11	comerciante_premium puede crear hasta 5 locales	El rol premium crea 5 locales permitidos y el sexto recibe 403.	Aprobada
12	local creado con varias imagenes devuelve la galeria	El local premium acepta una galería de varias imágenes y la devuelve en el detalle.	Aprobada
13	rol alcaldia no puede elegirse en el registro	El registro solo acepta turista/comerciante; el intento devuelve 400.	Aprobada
14	alcaldia solo puede crear un local y requiere su municipio	El rol alcaldia requiere id_municipio; solo publica 1 local (403 al segundo).	Aprobada
15	el local de la alcaldia aparece primero en la busqueda de su municipio	El local con destacado=1 se ordena antes que los de comerciantes en el listado.	Aprobada
16	eventos requiere autenticacion (F01)	GET /api/eventos sin token devuelve 401; con token responde 200 (SSE).	Aprobada

#	Prueba	Qué verifica	Resultado
17	publicacion con imagen_url de dominio externo es rechazada (F03)	imagen_url que no es Cloudinary ni /uploads responde 400.	Aprobada
18	publicacion acepta imagen de Cloudinary (F03)	imagen_url de res.cloudinary.com se acepta (201).	Aprobada
19	local con imagen_url de dominio externo es rechazado (F03)	El allowlist también aplica al crear locales (400).	Aprobada
20	upload limita la cantidad diaria por usuario (F02)	Al superar la cuota diaria según el rol la subida responde 429.	Aprobada
21	ciclo completo de publicacion: crear y eliminar	POST/PUT/DELETE de publicaciones como autor.	Aprobada
22	filtros de locales: municipio sin locales devuelve vacío	Filtros del listado de locales.	Aprobada
23	filtros de locales: departamento filtra sin aumentar resultados	Filtros del listado de locales.	Aprobada
24	origen CORS desconocido no recibe header Allow-Origin	Seguridad CORS por lista blanca.	Aprobada
25	origen CORS autorizado recibe su origen en Allow-Origin	El frontend autorizado puede consumir la API.	Aprobada
26	upload rechaza archivo con firma inválida	Se valida que el archivo sea una imagen real (400).	Aprobada
27	upload acepta un JPEG genuino	Subida correcta a Cloudinary con un JPEG real de prueba.	Aprobada
28	flujo completo de denuncia: crear, listar como admin y resolver	Denuncia (201), actualización de motivo sin duplicar, solo admin lista (403 al denunciante), resolución y estado inválido (400).	Aprobada
29	refresh sin cookie devuelve 401	La renovación de sesión exige la cookie httpOnly.	Aprobada
30	forgot-password responde idéntico si el email existe o no	No se revela si un correo está registrado.	Aprobada
31	reset-password rechaza tokens inválidos y expirados	Solo se acepta un token válido y vigente.	Aprobada
32	flujo completo de reset de contraseña: registrar, olvidar, restablecer, login	Todo el recorrido de recuperación (incluye reuso del token → 400).	Aprobada
33	rechaza registro con email duplicado	Integridad del email único.	Aprobada
34	flujo completo de refresh token con rotación y revocación	El refresh rota en cada uso; el logout revoca la sesión.	Aprobada

Importante: las pruebas 26 y 27 corresponden a la validación de imágenes reales (magic bytes) corregida en una versión anterior; las pruebas 10-15 cubren los roles comerciante_premium y alcaldia; las pruebas 16-20 cubren los hallazgos F01-F03 de la auditoría de seguridad.

4.1 Nueva prueba e2e de sesión (regresión)

Como parte de esta versión se detectó y corrigió un problema funcional relevante: en el login del frontend la petición no enviaba credentials (credentials: 'include'), por lo que la cookie httpOnly del refresh token no se guardaba y la sesión se perdía al recargar la página. Se corrigió en frontend/src/pages/Login.jsx y se añadió la prueba e2e «la sesión persiste al recargar la página (cookie httpOnly)» como regresión.

5. Pruebas e2e del frontend por caso

Se ejecutan en el bundle de producción compilado con Vite, contra el backend real, en Chromium headless.

#	Prueba	Qué verifica	Resultado
1	la home carga con la marca y la navegación	La portada muestra "REQUINTU" y los enlaces Locales y Publicaciones.	Aprobada

#	Prueba	Qué verifica	Resultado
2	locales muestra el buscador	La página de locales carga con su caja de búsqueda.	Aprobada
3	publicaciones muestra el muro	El muro carga y pide iniciar sesión para publicar.	Aprobada
4	la política de privacidad está disponible sin extensión	/politica-de-privacidad muestra su contenido.	Aprobada
5	la política de privacidad está disponible con extensión .html	/politica-de-privacidad.html también funciona.	Aprobada
6	una ruta protegida redirige al login cuando no hay sesión	Sin sesión, /perfil lleva a /login.	Aprobada
7	registro, inicio de sesión, cierre de sesión y limpieza del usuario	Flujo completo del usuario por la interfaz (el usuario de prueba se borra al final).	Aprobada
8	la sesión persiste al recargar la página (cookie httpOnly)	Tras iniciar sesión y recargar, la sesión se conserva gracias a credentials: include + refresh token en cookie httpOnly (regresión corregida en la v2.1).	Aprobada
9	el mapa aparece solo con sesión iniciada	Autenticación vía API, recarga y navegación al mapa con SVG visible.	Aprobada
10	el muro avisa en tiempo real cuando llega una publicación nueva	Dos sesiones: el observador ve el aviso de publicación nueva cuando el otro publica (canal SSE autenticado).	Aprobada
11	las publicaciones XSS se muestran como texto inerte en el muro	Un <script> y un en una publicación no se ejecutan: React los escapa.	Aprobada
12	los comentarios XSS en un local se muestran como texto inerte	Un <svg onload> y un en una calificación no se ejecutan.	Aprobada
13	denunciar una publicación desde el muro	Un usuario denuncia una publicación ajena desde el muro con motivo y detalle, y recibe la confirmación.	Aprobada

6. Tipos de prueba cubiertos

- Pruebas de integración (backend): la API se prueba completa contra bases de datos y servicios reales (MySQL/TiDB, Cloudinary, Brevo).
- Pruebas e2e (frontend): el usuario real recorre la aplicación en un navegador automatizado.
- Pruebas de seguridad: CORS, 401/403 sin token, tokens falsificados, anti-fuerza bruta, rotación y revocación de sesiones, filtro de roles, SSE autenticado (F01), cuota de subidas (F02), allowlist de imagen_url (F03) y XSS que se renderiza como texto inerte.
- Pruebas de validación: entradas inválidas (email, contraseña corta, archivo no-imagen, email duplicado).
- Pruebas de flujo: registro, login, logout, recuperación de contraseña, publicaciones CRUD, subida de imágenes, persistencia de sesión al recargar.
- Pruebas de roles: límites por rol (comerciante 1 local, comerciante_premium 5 locales con galería, alcaldía 1 local destacado), asignación de roles solo por admin.
- Pruebas de tiempo real: verificación del aviso del muro vía SSE autenticado entre dos sesiones.
- Pruebas de moderación de la comunidad: clasificador NSFW local en la subida de imágenes y flujo de denuncias con gestión por el administrador.

7. Resultado final

Métrica	Valor
Backend aprobadas	34/34
E2E aprobadas	13/13
Tasa de éxito	100%

Todas las pruebas fueron ejecutadas el 18 de septiembre de 2026 y quedaron versionadas en el repositorio (rama main) dentro de test/api.test.mjs y frontend/e2e/.

Fin del informe de pruebas.